System Design Documentation

JS Law Group

24/7 Legal Rescue Website

Jaehoon Song

February 18, 2026

Contents

1 Requirements: Functional and Non-Functional

This document captures the baseline functional and non-functional requirements for the JS Law Group marketing website. It serves as the authoritative guide for all development, quality assurance (QA), and future enhancement work.

1.1 Scope

The scope of this system covers the public-facing, multilingual web application for clients in emergency situations. The website contains minimal contact and law firm information (e.g., practice area summary, address, social links, emergency call, wizard) so users can quickly request help. Content presentation focuses on practice areas and the client contact intake workflow via the Auto Accident Wizard. The application is implemented as a Python Flask backend with server-side form processing, validation (Flask-WTF), and email dispatch (GmailProxy).

Payment processing, user authentication, and internal case management systems are intentionally out of scope for this project.

1.2 Functional Requirements

1.2.1 Site Shell and Navigation

1. **FR-01: Consistent Shell.** The system **shall** display a consistent navigation header and footer across all sections of the site.
2. **FR-02: Fixed Navigation.** The main navigation menu **shall** remain fixed at the top of the screen. It **shall** provide links to all main content sections and **shall** collapse into a mobile-friendly menu icon on small screens.
3. **FR-03: Landing Hero.** The main landing section **shall** display a translated headline, a subtitle, and summary cards for the main practice areas (Personal Injury, Criminal Defense, Family Law). The section **shall** also provide three separate, independent elements: (1) a call-to-action button, (2) a main website button or link, and (3) the Accident Wizard, which **shall** be present in the hero section. These three **shall** not depend on each other.

1.2.2 Localized Content Delivery

1. **FR-04: Locale Management.** The system **shall** manage translations using JSON-based locale files (e.g., `en-US.json`, `ko-KR.json`, `ja-JP.json`, `es-US.json`) in a central `static/locales/` (or equivalent) directory. English **shall** be the default fallback language.
2. **FR-05: Runtime Switching.** Language options **shall** be provided on the Floating Action Button (FAB). Selecting a language **shall** immediately translate all text on the page via the i18n system and **shall** persist the user's choice (e.g., in LocalStorage) for future visits.
3. **FR-06: Translatable Content.** All visitor-facing text **shall** be sourced from the central translation files and **shall** be wired for translation (e.g., via `data-i18n` or equivalent). This includes articles, labels, buttons, and form fields.
4. **FR-07: State Refresh.** The system **shall** refresh its state when the browser tab regains focus, ensuring the most recent content and language setting are displayed.
5. **FR-08: Practice Detail Sections.** The site **shall** feature dedicated sections for Personal Injury and Criminal Defense, showing relevant imagery and bulleted lists of services. Content is kept minimal for emergency-focused users.

1.2.3 Engagement and Intake

1. **FR-09: Independent Engagement Controls.** The system **shall** provide a call-to-action button, a main website button (or link) to the main firm website, and the Accident Wizard (in the hero section) as separate, independent controls. None of these **shall** depend on the others; each **shall** function on its own.
2. **FR-10: Accident Wizard Form.** The Auto Accident Wizard **shall** be the sole form for submitting case inquiries. It **shall** collect accident type, details (e.g., ZIP code, medical treatment, police report status), and contact information (name, email, phone) in a multi-step flow. Form handling **shall** use Flask-WTF/WTForms with CSRF protection and server-side validation.
3. **FR-11: Backend Email Dispatch.** Upon wizard submission, the system **shall** validate inputs and dispatch a structured inquiry email to the firm via the configured Gmail SMTP service (using `GmailProxy`), with attachments (e.g., JSON, CSV, HTML) as implemented. Submissions **shall** be stored on the server in a controlled `submissions/` directory for intake and audit purposes. The system **shall** send a phone/SMS follow-up to the client via the Twilio API right after submission (incident follow-up). The system **shall** register submitters (and optionally relatives or close contacts) with the Mailchimp API for the law firm's marketing and advertise email campaigns (see 2.4.7).
4. **FR-12: Floating Action Button.** A "Floating Action Button" (FAB) **shall** be visible on mobile for switching languages (e.g., opening the language selector).
5. **FR-13: Contact Section.** The contact section **shall** display the firm's address and hyperlinks to the firm's official Facebook and Instagram only. It **shall** not include a map, phone links, or email links.
6. **FR-14: Social Links.** The footer **shall** provide links to the firm's verified social media profiles (e.g., Facebook, Instagram) and static policy pages.

1.2.4 Backend Logic (Python Flask)

1. **FR-15: Routing.** The application **shall** use Flask to route URL requests to appropriate view functions (e.g., `/`, `/motor-vehicle-accident`).
2. **FR-16: Form Validation.** The system **shall** use `Flask-WTF` and `WTForms` to validate all user inputs server-side, ensuring required fields and data formats (e.g., email, phone) are correct.
3. **FR-17: CSRF Protection.** All forms **shall** include a CSRF token to prevent Cross-Site Request Forgery attacks.
4. **FR-18: OS Detection.** The `main.py` wrapper **shall** automatically detect the operating system:
   - **Windows:** Launch `Waitress` WSGI server.
   - **Linux/macOS:** Launch `Gunicorn` WSGI server.

1.2.5 Email Automation Service

1. **FR-19: SMTP Integration.** The `GmailProxy` service **shall** authenticate with a Gmail SMTP server using secure app passwords.
2. **FR-20: Data Persistence.** Upon form submission, the system **shall** save the data locally in `submissions/` as:
   - `.json` (raw data object)
   - `.csv` (comma-separated values)
   - `.html` (formatted body)
3. **FR-21: Attachment Handling.** The system **shall** attach the generated JSON and CSV files to the notification email sent to the firm.

1.2.6 Frontend & Internationalization (i18n)

1. **FR-22: Bilingual Implementation.** The system **shall** support dynamic switching between English (en-US), Korean (ko-KR), Japanese (ja-JP), and Spanish (es-US) using a custom JSON-based i18n module.
2. **FR-23: Responsive Implementation.** The UI **shall** utilise Bootstrap 5 to ensure full responsiveness across mobile, tablet, and desktop devices.
3. **FR-24: Debug Mode.** The system **shall** provide a visual debug mode to highlight translated elements for developers.

1.3 Non-Functional Requirements

1. **NFR-01: Responsive Layout.** The website layout **shall** be fully responsive using a mobile-first approach, adapting cleanly to desktop, tablet, and mobile screen sizes without horizontal scrolling.
2. **NFR-02: Accessibility.** The website **shall** adhere to Web Content Accessibility Guidelines (WCAG) AA standards. This includes use of semantic HTML5, alt text for images, and full keyboard navigation support.
3. **NFR-03: Performance.** The website **shall** load quickly. External resources (fonts, icons) **shall** be loaded from a Content Delivery Network (CDN). The Largest Contentful Paint (LCP) **shall** be under 2.5 seconds on a standard broadband connection. Frontend **shall** use a modular JavaScript architecture and optimized asset delivery as documented in the project.
4. **NFR-04: Privacy and Security.** Personally Identifiable Information (PII) **shall** only be collected via the Accident Wizard form and **shall** be handled in accordance with the firm's privacy policy. The system **shall** store submissions only in the controlled `submissions/` directory and transmit them to the firm via Gmail SMTP. The system **shall** only use local browser storage for language preferences. A privacy statement **shall** be included in the footer.
5. **NFR-05: Localization Quality.** When new content is added, corresponding translations **shall** be added for all supported languages to avoid displaying fallback (English) text.
6. **NFR-06: Maintainability.** All new code **shall** be modular, separated from content (e.g., translations in JSON, templates in `templates/`, styles in `static/`), and well-documented to ensure future maintainability.
7. **NFR-07: Availability and Hosting.** The system **shall** be implemented as a Python Flask application. It **shall** be hostable using a production WSGI server: Waitress on Windows and Gunicorn on Linux/Mac. The application **shall** support optional bundling as a standalone executable (e.g., via PyInstaller) for target platforms.
8. **NFR-08: Browser Support.** The system **shall** be fully functional on the two most recent versions of Chrome, Safari, Edge, and Firefox on both desktop and mobile platforms.

1.4 Dependencies and Constraints

- The system **is constrained** to use the Bootstrap 5 framework for layout and responsive behavior.
- The system **is constrained** to use the Font Awesome 6 icon library for UI icons.
- The system **is dependent** on the Google Fonts service for typography (e.g., Inter, Noto Sans KR as documented in the project).
- The system **is dependent** on Python Flask, Flask-WTF/WTForms for form handling and CSRF protection, and a production WSGI server (Waitress or Gunicorn) for deployment.
- The contact workflow **is dependent** on the backend having valid Gmail SMTP credentials (for `GmailProxy`) and outbound network access to the Gmail service.

- The system **is dependent** on the user's browser supporting LocalStorage for persisting language preferences.
- The system **is dependent** on the Twilio API for sending a phone/SMS follow-up to the client right after wizard submission (incident follow-up).
- The system **is dependent** on the Mailchimp API to register inquiry submitters (and optionally relatives or close contacts) for the law firm's marketing and advertise email campaigns.

2 Behavioral Specification: Use Case Analysis

This section describes the behavioral specification of the 24/7 Legal Intake Website through use case analysis. It identifies stakeholders and actors, presents a use case diagram, and provides fully dressed use case descriptions for development and QA.[1]

2.1 Stakeholders

Stakeholder	Goals and Concerns
Prospective Clients	Need fast access to practice area information, bilingual copy, and an immediate way to request help (emergency call or wizard) without sharing payment or sensitive data online.
Family or Community Referrals	Want minimal firm information (office address, social links) on behalf of someone in need.
Attorneys and Intake Staff	Need complete, well-formatted inquiries routed to the firm's intake email so follow-ups can be triaged quickly.
Marketing/Admin Team	Owns site content, translations, imagery, and compliance statements; cares about maintainable assets and accurate brand voice.
Technology Operations	Maintains the website hosting and ensures all website dependencies remain reachable and secure.

2.2 Actors

2.2.1 Primary Actors

- **Prospective Client (English).** A person seeking legal representation who speaks English.
- **Prospective Client (Bilingual).** A person seeking legal representation who prefers to read in Korean, Japanese, or Spanish.
- **Referral or Family Member.** A person collecting information (e.g., address, social links) on behalf of someone else.

2.2.2 Secondary Actors (External Systems and APIs)

- **Third-Party Content Services.** External hosts that the system relies on to provide fonts, icons, and other design assets.
- **Gmail SMTP Service.** The email transport service used by the backend to dispatch inquiry notifications and wizard submissions.
- **Twilio API.** Messaging service used by the backend to send a phone/SMS follow-up to the client right after they submit the Accident Wizard form (incident-related follow-up).
- **Mailchimp Marketing API.** External service used to register inquiry submitters (and optionally their relatives or close contacts) so the law firm can send them marketing and advertise emails for follow-up and ongoing connection.

[1] **Activity diagrams** and **communication diagrams** have been omitted, as the system does not have the complexity that would warrant those diagrams.

2.3 Use Case Diagram

Figure 1 shows actors and use cases for the 24/7 Legal Intake Website.

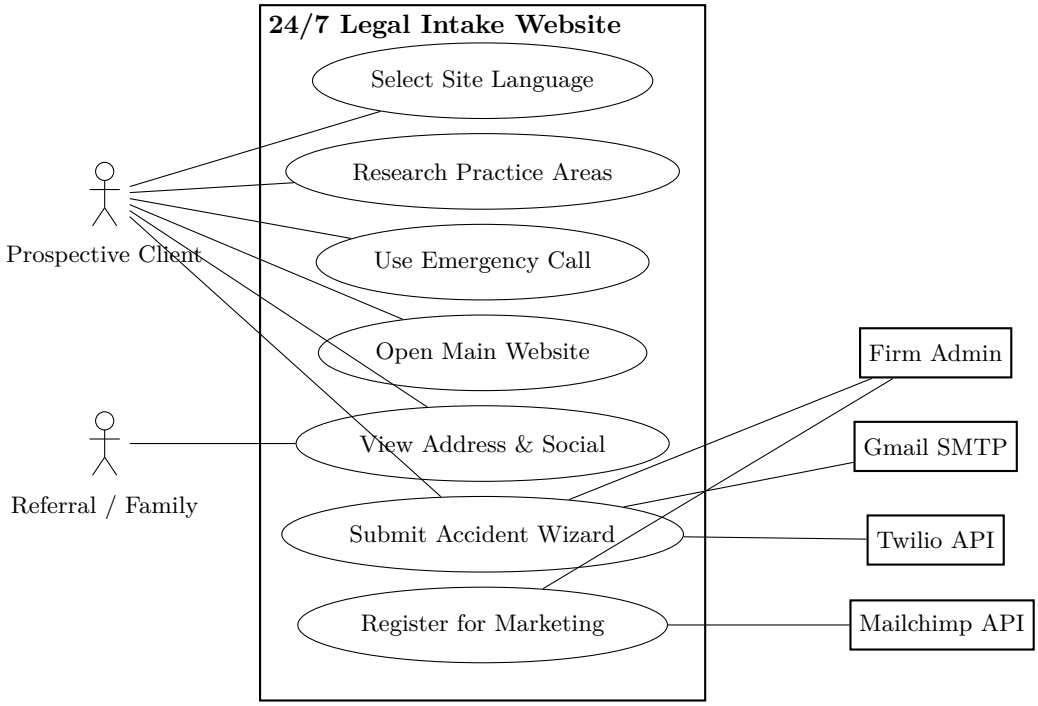

Figure 1: UML Use Case Diagram - actors and use cases for the 24/7 Legal Intake Website

2.4 Use Cases (Fully Dressed)

2.4.1 UC-01: Select a Site Language

Primary Actor: Prospective client (bilingual)

Secondary Actors: User's Browser

Goal: View all copy in a preferred language to evaluate the firm comfortably.

User Story: As a Korean-speaking client, I want to flip the site into Korean so I can confirm services and contact steps without translation mistakes.

Preconditions: The website has fully loaded in the default language.

Trigger: The visitor opens the FAB and clicks one of the language options.

Main Scenario:

1. The actor activates the Floating Action Button (FAB) to open the language selector.
2. The actor selects a locale option (e.g., "Korean" or "Spanish" or "Japanese").
3. The system retrieves the corresponding text and updates every label, button, and article on the page.
4. The system saves this language choice in the user's browser, so it is remembered on future visits.

Alternate Flows:

- If the translation file cannot be loaded (e.g., network disruption), the system logs an error, and the website remains in the previous language.

Postconditions: The chosen language persists for the session. All subsequent actions and popups are shown in the selected language.

2.4.2 UC-02: Research Practice Areas

Primary Actor: Prospective client (any language)

Secondary Actors: Third-Party Content Services (for images)

Goal: Understand whether JS Law Group covers the specific legal scenario at hand.

User Story: As someone recovering from a collision, I want to skim the Personal Injury section and content to determine if the firm handles car and premises cases.

Preconditions: The website is loaded.

Trigger: The actor lands on the site or scrolls / clicks "Practice Areas".

Main Scenario:

1. The actor glances at the main service cards to confirm the firm covers PI, Criminal Defense, and Family Law.
2. The actor scrolls to the Personal Injury section, reviewing the translated bullet list of featured services and any minimal copy.
3. If relevant, the actor repeats the process with the Criminal Defense section.
4. The actor may use the emergency call or the Accident Wizard if ready to proceed.

Alternate Flows:

- On mobile, the actor may use the main menu to jump directly to practice sections or to the wizard.

Postconditions: The actor has enough information to decide whether to contact the firm (e.g., via emergency call or wizard).

2.4.3 UC-03: Use Emergency Call Button

Primary Actor: Prospective client ready to contact

Secondary Actors: User's Phone/Voice Application

Goal: Immediately place a phone call to the firm in an urgent situation.

User Story: As a client in an emergency, I want one tap to call the firm so I can speak to someone right away.

Preconditions: The emergency call button is visible on the screen (mobile or desktop).

Trigger: The actor clicks or taps the emergency call button.

Main Scenario:

1. The actor activates the emergency call button.
2. The system invokes the device's dialer with the firm's phone number pre-filled.
3. The actor confirms or places the call.

Alternate Flows:

- If the device cannot place calls (e.g., no telephony capability), the system shows the firm's phone number so it can be dialed from another device.

Postconditions: The actor is in a position to speak with the firm by phone, or has the number needed to do so.

2.4.4 UC-04: Open Main Firm Website

Primary Actor: Prospective client wanting more background about the firm

Secondary Actors: Web browser

Goal: View the full JS Law Group website for deeper information (e.g., history, additional services).

User Story: As a client researching the firm, I want a clear button that takes me to the main website so I can read more before deciding to call.

Preconditions: The redirect button or link to the main website is visible.

Trigger: The actor clicks the "Visit Main Website" button.

Main Scenario:

1. The actor clicks the redirect button.
2. The system opens the main JS Law Group website in the same tab or a new tab, as configured.

Alternate Flows:

- If the main website is temporarily unavailable, the system shows an error message or fallback contact information.

Postconditions: The actor can browse the main firm website for additional details about the practice.

2.4.5 UC-05: View Address and Social Links

Primary Actor: Referral / family member confirming logistics

Secondary Actors: Web browser

Goal: Obtain the office address and hyperlinks to the firm's official Facebook and Instagram.

User Story: As a friend helping someone find the firm, I want to see the address and links to the firm's official social profiles so I can share them.

Preconditions: The website is loaded.

Trigger: The actor scrolls or navigates to the contact section.

Main Scenario:

1. The actor scrolls to or navigates to the contact section.
2. The actor reviews the firm's address (plain text).
3. The actor uses the hyperlinks to open the firm's official Facebook and Instagram profiles.

Alternate Flows:

- If the contact section cannot be loaded correctly, the system still displays at least the address and social links in plain text.

Postconditions: The actor has the office address and can access the firm's official Facebook and Instagram pages.

2.4.6 UC-06: Submit Auto Accident Wizard

Primary Actor: Prospective client.

Secondary Actors: Email Service Provider (Gmail); Firm Administrator; Twilio API (phone/SMS incident follow-up).

Goal: Complete a multi-step assessment to report a motor vehicle accident.

Preconditions: The user is on the homepage; the Accident Wizard is present in the hero section and visible.

Trigger: The user interacts with the "Accident Wizard" form.

Main Scenario:

1. The user selects the accident type ("Car" or "Motorcycle" or etc.) and provides accident details (ZIP code, medical treatment, police report status, and related fields).
2. The user enters contact information (name, email, phone).
3. The system validates all inputs (CSRF token, email format, and related fields).
4. The system dispatches a structured email to the Firm Administrator via Gmail SMTP with the JSON/CSV attachments after processing the form via Flask-WTF.
5. The system sends a phone/SMS follow-up to the client via the Twilio API (incident follow-up right after submission).
6. The system displays a success confirmation to the user.

Postconditions: An inquiry email with attachments is delivered to the Firm Administrator; the user sees a confirmation message; the client receives a phone/SMS follow-up via Twilio.

2.4.7 UC-07: Register for Marketing Email (Mailchimp)

Primary Actor: System (automated when inquiry data is available) or Firm Administrator.

Secondary Actors: Mailchimp Marketing API.

Goal: Register inquiry submitters (and optionally relatives or close contacts) in Mailchimp so the law firm can send them marketing and advertise emails (e.g., annual campaigns and follow-up).

Preconditions: Wizard submission data has been received and stored (e.g., in `submissions/` or intake system).

Trigger: New inquiry data is available for registration, or the Firm Administrator initiates sync to Mailchimp.

Main Scenario:

1. The system (or administrator) sends the submitter's contact information (and optionally relatives or close contacts) to the Mailchimp API to add to the law firm's audience/list.
2. Mailchimp registers the contact(s) for the firm's marketing and advertise email campaigns.

Postconditions: The submitter (and optionally relatives or close contacts) is registered in Mailchimp; the law firm can send them marketing and annual advertise emails.

2.5 Sequence Diagrams

The following sequence diagram captures the primary legal intake submission flow from the user's perspective through to backend persistence and notification. It shows the main participants—User, Browser, Flask App, File System, and Gmail SMTP—and the order of interactions: the user accesses the site and receives the wizard (GET `/`), fills the 10-step Auto Accident form, and submits (POST `/`); the server validates input (e.g. via WTForms), saves the submission as JSON/CSV/HTML under `submissions/`, and sends a notification email via Gmail SMTP before returning a success response to the browser. Figure 2 illustrates this flow.

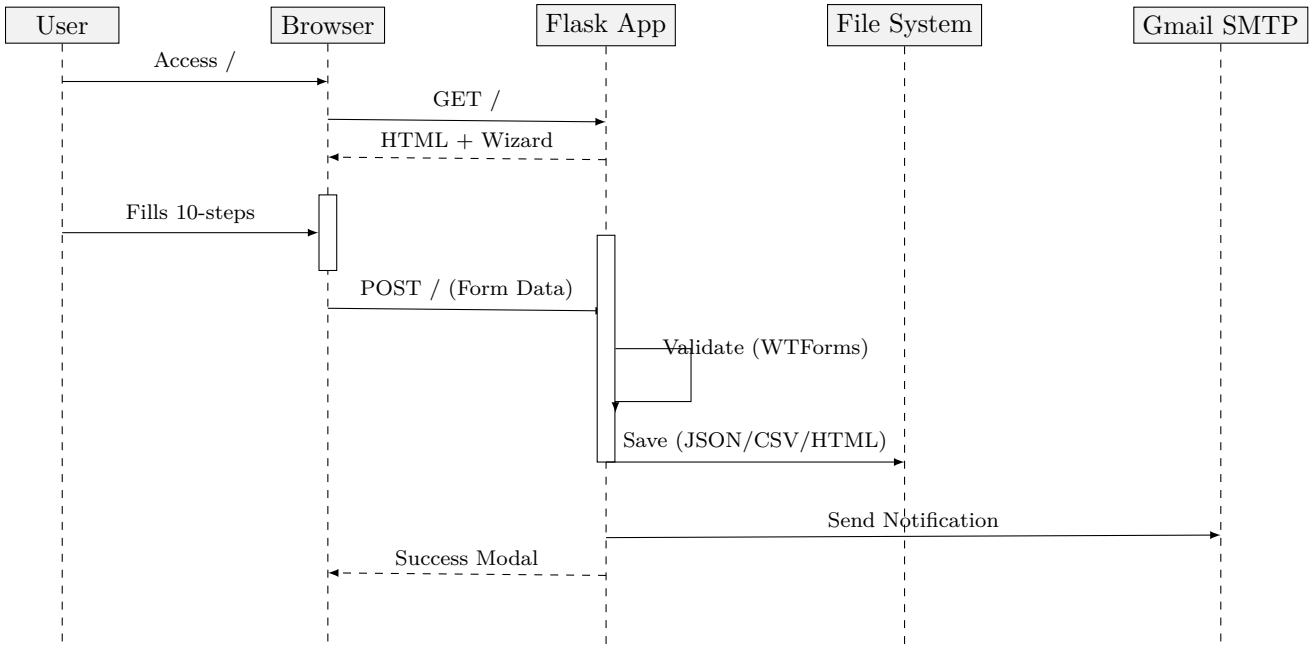

Figure 2: UML Sequence Diagram - Legal Intake Submission Flow

2.6 State Diagram (UI Flow Diagram)

The diagram in Figure 3 illustrates the navigation paths and user interaction flow within the application.

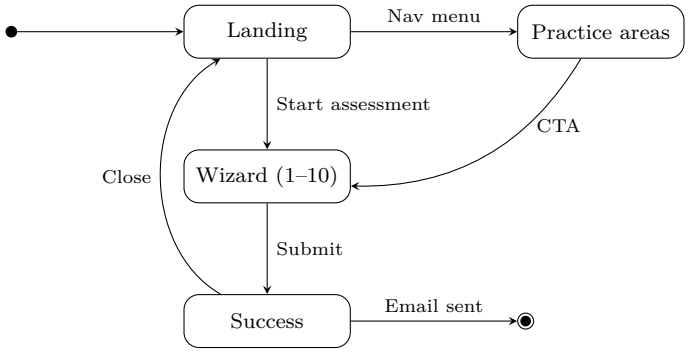

Figure 3: UML Activity Diagram - UI flow

2.6.1 User Interface States

- **Landing State:** Users see the home page with the Auto Accident Wizard in the hero section, plus call-to-action, main-website link, and practice-area summary (FR-03 in 1). The Floating Action Button (FAB) provides language selection (`en-US`, `ko-KR`, `ja-JP`, `es-US`) with persistence in LocalStorage (2.4.1).
- **Wizard State:** A single, progressive 10-step form (accident type, ZIP, medical treatment, police report, prior attorney, accident date, injury type, name, contact details, submit). It is the sole form for case inquiries (FR-10 in 1). Practice area pages link back to the wizard (e.g. `#wizard-container`) for inquiry CTAs.
- **Practice Area State:** Dedicated routes for `/motor-vehicle-accident` and `/personal-injury`. The Personal Injury page includes a Criminal Defense section; there is no separate Criminal Defense route. Content is minimal and i18n-enabled (FR-08 in 1).
- **Submission State:** After server-side validation (Flask-WTF, CSRF), the user sees a success modal; data is written to `submissions/` (JSON, CSV, HTML) and an email is sent via `GmailProxy`. Twilio follow-up and Mailchimp registration are as specified in 2.4.6 and 2.4.7.

3 Structural Specification: System Architecture

The JS Law Group's 24/7 Legal Intake Website application follows a Model-View-Controller (MVC) pattern adapted for Flask, deployed as a containerized application on cloud infrastructure.[2] This section aligns with the functional and non-functional requirements (1), use cases (2), and the UI flow (3).

3.1 System Architecture Diagram

Figure 4 shows the application components and their interfaces; Figure 5 shows the deployment topology on OCI.

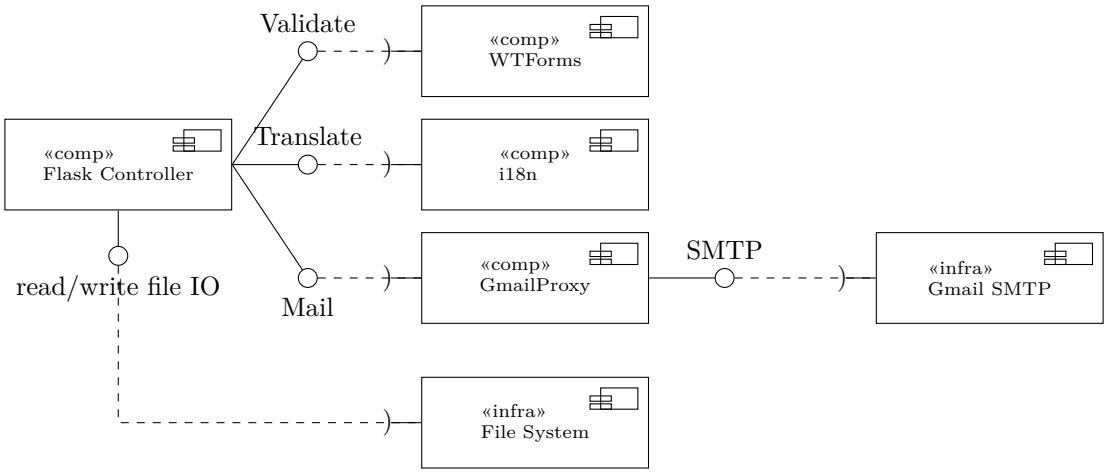

Figure 4: UML Component Diagram - application modules and external interfaces

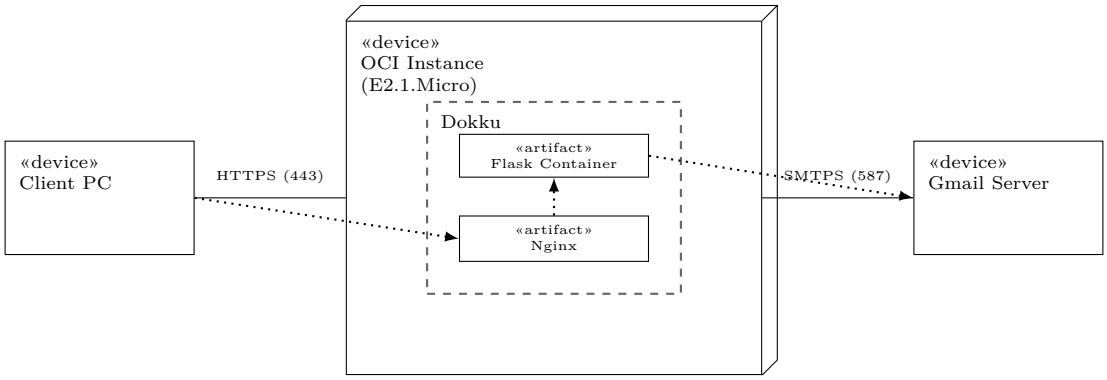

Figure 5: UML Deployment Diagram - OCI instance with Dokku, Flask container, and Nginx

3.2 Application Stack and Data Flow

The application uses Flask for routing and request handling. Form submission is the sole intake path: the Auto Accident Wizard (Flask-WTF/WTForms with CSRF protection) posts to `/`. On success, the backend saves each submission under `submissions/` as three files (JSON, CSV, and HTML body), then sends a notification email to the firm via `GmailProxy` with JSON and CSV attached. The system also sends a post-submission SMS follow-up via Twilio and registers submitters with Mailchimp for marketing email campaigns, as documented in 2.4.6 and 2.4.7.

[2] **Class diagram**, **Object diagram**, and **Package diagram** have been omitted, as the system does not have the complexity that would warrant those diagrams.

Key routes:

- `/` (GET/POST: landing and wizard)
- `/motor-vehicle-accident`
- `/personal-injury`
- `/robots.txt`
- `/sitemap.xml`

Internationalization is provided by a custom JSON-based i18n module with locale files in `static/locales/` (e.g. `en-US.json`, `ko-KR.json`, `ja-JP.json`, `es-US.json`); the Floating Action Button (FAB) allows runtime language switching with persistence in LocalStorage.

3.3 Infrastructure Components

- **IaaS Provider:** Oracle Cloud Infrastructure (OCI) - Instance Shape: `VM.Standard.E2.1.Micro` (AMD EPYC, 1GB RAM).
- **PaaS Layer:** Dokku (v0.34.6) providing Heroku-like deployment workflows (Git push to deploy).
- **OS:** Ubuntu Linux.
- **WSGI:** Gunicorn on Linux/macOS; Waitress on Windows (auto-detected by the application launcher).
- **CI/CD:** GitHub Actions pipeline for building standalone executables via PyInstaller for target platforms.

3.4 Third-Party Integrations

Service	Usage
Gmail SMTP	Outbound transactional emails for case inquiries (via `GmailProxy`).
Twilio API	Phone/SMS follow-up to the client right after wizard submission (2.4.6).
Mailchimp API	Register inquiry submitters for marketing email campaigns (2.4.7).
FontAwesome	Iconography delivery via CDN.
Google Fonts	Typography assets (e.g. Plus Jakarta Sans, as documented in the project).

The contact section does not include a map, per requirement FR-13 in 1.